

Manual de rol — Persona B

Conducta y Reportes

Versión 2.0 · 9 de septiembre de 2026 · Escrito por Persona C
Cierre del Shipaton: **30 de septiembre, 23:45 PDT** — quedan 21 días.

Este manual reemplaza al de agosto para tu rol. Aquel describía el stack retirado (PostgreSQL, Drizzle, RLS, Better Auth) y seguirlo hoy te haría reconstruir cosas que el proyecto abandonó el 16 de agosto. Está archivado en <docs/99-archivo/>. Este es el vigente: Convex + Clerk, y solo lo que a ti te falta.

Para qué sirve este documento

Para que en **una semana** termines todo lo que le falta a tu rol, sin tener que preguntar por dónde seguir. Cada tarea viene con el paso a paso, el código de ejemplo y cómo comprobar que quedó bien.

Está escrito con lo que hay **hoy** en el repositorio, verificado archivo por archivo — no de memoria. Donde tus issues de GitHub dicen algo que ya no es cierto, aquí está corregido y señalado.

Tu rol en una frase

Construyes el corazón funcional del producto.

El puntaje, las acciones, la asistencia y los reportes son tu módulo. Todo lo que hace que Cresco sea Cresco y no otra app de mensajería pasa por ahí. Eres quien más lógica de negocio escribe y quien más pruebas necesita, porque un error tuyo no se ve como un error: se ve como un estudiante con el puntaje equivocado, y eso es exactamente lo que un padre va a reclamar.

Y ahora mismo eres la ruta crítica del proyecto. Sin `conducta.ts` no existe el puntaje, no existen los reportes, y no se pueden construir 10 de las 31 pantallas. Daniel terminó lo suyo, Kenny también. Tu módulo es lo único que separa al equipo de tener un producto completo.

Índice

1. **Parte 1** — Dónde estás parado hoy
2. **Parte 2** — Tu alcance: qué es tuyo y qué no
3. **Parte 3** — Día 0: dejar la máquina lista (45 minutos)

4. **Parte 4** — Las cinco reglas que no se negocian
5. **Parte 5** — Cómo escribir una prueba (el andamio que vas a copiar 20 veces)
6. **Parte 6** — Día 1: cerrar el issue #9
7. **Parte 7** — Día 2: asistencia
8. **Parte 8** — Días 3 y 4: el reporte del día
9. **Parte 9** — Día 5: las tres consultas del representante
10. **Parte 10** — Día 6: comunicados y cron nocturno
11. **Parte 11** — Día 7: lo que los demás necesitan de ti
12. **Parte 12** — Cómo se entrega: rama, PR, CI
13. **Apéndices** — Tablas, guardas, reglas, semillas, errores comunes

Parte 1 — Dónde estás parado hoy

1.1 Lo que ya escribiste, y está bien

El PR #39 tiene `registrarAccion` funcionando. Lo revisé a fondo el 6 de septiembre y te lo digo otra vez porque importa: **el código está bien pensado.**

- **Respetaste ADR-005**, que es la regla más fácil de romper. No incrementas el puntaje: lo recalculas desde las acciones `VIGENTE` del período.
- **No escribiste ni un valor a mano.** Los límites salen de `tipoAccion`; la base, el piso y el techo salen de `institucion`.
- **El tope diario está bien resuelto, y no era obvio.** Sumas solo los puntos previos *del mismo signo*. Si hubieras sumado todos, una acción positiva del día habría compensado una negativa y el tope se calcularía mal.
- **Respetas `congelado`**: si el período ya cerró, no tocas el puntaje.

1.2 Lo que falta, y por qué está parado

El PR #39 lleva **un solo commit desde el 6 de septiembre** y sigue con cambios pedidos. Lo que bloquea la aprobación:

Falta	Dónde	Tiempo estimado
Las tres pruebas del issue #9	<code>conducta.test.ts</code> (no existe todavía)	2–3 horas
<code>anularAccion</code>	<code>conducta.ts</code>	1 hora
Tres <code>throw new Error(...)</code> sueltos → <code>ErrorDominio</code>	<code>conducta.ts</code>	10 minutos

Nada de eso es difícil. Es media jornada. Lo que falta es empezar.

1.3 El plan de la semana

Día	Qué entregas	Cierra
0 (hoy, 45 min)	Despliegue de Convex funcionando con semillas	#8
1	Pruebas + <code>anularAccion</code> + errores públicos + auditoría	#9
2	Asistencia	—
3 y 4	Reporte general y reportes por estudiante	—
5	Las tres consultas del representante (P4, P5, P6)	desbloquea las pantallas
6	Comunicados (notas y eventos) + cron nocturno	—
7	Recálculo expuesto para Kenny + repaso final	#10

1.4 Quién está esperando exactamente qué

Quién	Qué espera	Para
Kenny (C)	Que <code>conducta.ts</code> llame a <code>auditar()</code> en <code>CREAR</code> y <code>ANULAR</code>	Cerrar #19 — es el último de los cuatro eventos de DP-006
Kenny (C)	Una función de recálculo exportada	El <code>RECALCULO_PENDIENTE</code> de <code>interaccion.ts</code> : cuando un reclamo se resuelve como <code>MODIFICADA</code> , hay que devolverle los puntos al estudiante
Interfaz (D)	<code>registrarAccion</code> , asistencia y reporte general	Pantallas D11–D14 (#31)
Interfaz (D)	Las tres consultas del representante	Pantallas P4, P5 y P6 (#32) — P4 es la pantalla más importante del producto

Parte 2 — Tu alcance: qué es tuyo y qué no

2.1 Tus archivos

Archivo	Estado	Qué va ahí
<code>movil/convex/conducta.ts</code>	existe, a medias	Todas tus mutations y queries
<code>movil/convex/conducta.test.ts</code>	no existe — lo creas tú	Todas tus pruebas
<code>movil/convex/crons.ts</code>	existe (PR #46)	Añades tu cron nocturno debajo del que ya está

2.2 Tus tablas

Eres dueña de estas. Nadie más escribe en ellas:

`categoriaAccion` · `tipoAccion` · `accionRegistrada` · `comunicadoCurso` · `franjaConducta` ·
`puntajePeriodo` · `registroAsistencia` · `plantillaReporte` · `plantillaCampo` · `reporteGeneral` ·
`reporteGeneralValor` · `reporteEstudiante` · `reporteEstudianteItem` · `entregaReporte`

El detalle campo por campo está en el **Apéndice A**.

2.3 Lo que NO tocas

Superficie compartida — cambiarla exige la aprobación de los tres, y GitHub la pide sola:

`convex/schema.ts` · `convex/lib/enums.ts` · `convex/lib/guardas.ts` · `convex/lib/permisos.ts` ·
`convex/lib/flags.ts`

Si necesitas un campo o una constante que no existe, **pídelo en el grupo**. No lo agregues por tu cuenta: el PR se va a quedar esperando tres aprobaciones y vas a perder un día.

Tampoco toques `convex/nucleo.ts` (Daniel) ni `convex/interaccion.ts` (Kenny). Puedes **leer** sus tablas — de hecho vas a leer `matricula`, `curso`, `periodoAcademico` e `institucion` todo el tiempo — pero no escribes en ellas.

2.4 Dos correcciones a tus issues de GitHub

Los issues #9 y #10 tienen dos firmas equivocadas. Las verifiqué contra el código de hoy. Si copias lo que dice el issue, no compila.

El issue dice	La firma real
<code>calcularPuntaje(base, suma, piso, techo)</code>	<code>calcularPuntaje(puntos[], base?, piso?, techo?)</code> El primer argumento es el arreglo de puntos de las acciones vigentes, no la base. Los otros tres son opcionales y ya traen los valores de REGLAS .
<code>exigirVinculo(ctx, representanteId, estudianteId)</code>	<code>exigirVinculo(ctx, estudianteId)</code> El representante sale de la sesión, no de un argumento. Es a propósito: si viniera por argumento, cualquiera podría pasar el id de otro.

Parte 3 — Día 0: dejar la máquina lista

45 minutos. Cierra el issue #8.

3.1 Los pasos

```
# 1. El repositorio al día
git checkout main
git pull
npm ci

# 2. Tu propio despliegue de Convex (se decidió uno por persona, no compartido)
cd movil
npx convex dev
```

`npx convex dev` te va a pedir iniciar sesión y crear un proyecto. Acepta. Al terminar escribe `movil/.env.local` con tu `CONVEX_DEPLOYMENT`. **Ese archivo no se sube a git** y no debe subirse nunca.

Déjalo corriendo en una terminal: vigila los cambios y los sube a tu despliegue solo.

```
# 3. En OTRA terminal, cargar los datos de prueba
cd movil
npx convex run semillas:cargar
```

3.2 Conectar Clerk

En el panel de Convex (`dashboard.convex.dev`) → tu proyecto → **Settings** → **Environment Variables**, agrega:

Variable	Valor
CLERK_JWT_ISSUER_DOMAIN	La <i>Frontend API URL</i> que muestra Clerk, del estilo <code>https://algo.clerk.accounts.dev</code>

Tiene que coincidir exactamente con el `iss` del token de Clerk. Una barra final de más y los perfiles dejan de resolverse en silencio: la gente vería «completa tu perfil» con su perfil intacto en la base. Si no la tienes a mano, pídesela a Kenny.

3.3 Comprobar que quedó bien

```
cd ..          # a la raíz del repositorio
npm run typecheck # tiene que pasar sin decir nada
npm test       # tiene que pasar en verde
```

Y en el panel de Convex → **Data**, deberías ver poblado:

- **6 franjas** de conducta, con color y frase

- **6 categorías** de acción
- **7 tipos** de acción
- **5 planes**
- **1 plantilla** de reporte con **4 campos**

Si eso está, cierra el issue #8 con un comentario que diga que tu despliegue quedó configurado.

`semillas:cargar` es idempotente: puedes correrlo las veces que quieras.

3.4 Lee esto antes de escribir código

`Contexto/CLAUDE.md` — las 11 reglas del proyecto y dónde vive cada cosa. Son 10 minutos y te ahorran discusiones.

Parte 4 — Las cinco reglas que no se negocian

Regla 1 — El puntaje se recalcula, nunca se suma (ADR-005)

```
puntaje = clamp(base + Σ puntos de acciones VIGENTES del período, piso, techo)
```

`puntajePeriodo` es una **caché reconstruible**. La verdad son las acciones. Si en algún punto te encuentras escribiendo `puntajeActual = puntajeActual - 1`, para: está mal.

La razón no es purismo. Una acción puede anularse o modificarse tras un reclamo, y en ambos casos pasa a valer 0. Si el puntaje fuera un contador, cualquier fallo lo desincronizaría de la bitácora de forma permanente **y sin manera de detectarlo**.

Usa siempre `calcularPuntaje()`. Recalcula en tres momentos: al registrar una acción, al anularla o modificarla, y al cerrar el período.

Regla 2 — No hay catálogo de nombres de acción (DP-002)

El docente elige una **categoría** y escribe el mensaje con sus propias palabras. Esto se decidió con un profesor real. No inventes una lista de acciones predefinidas.

Signo	Categorías	Rango de puntos
Positiva	Desempeño · Convivencia · Responsabilidad · Puntualidad	+1 a +2, a criterio del docente
Negativa	Indisciplina · Deshonestidad	-1 a -3, a criterio del docente
Negativa	Responsabilidad (irresponsabilidad)	-1 fijo , sin ajuste

Regla 3 — Los valores viven en el código, no en tu archivo

Todo sale de `REGLAS` en `convex/lib/enums.ts` (Apéndice C). Si escribes un `60` o un `-5` suelto en `conducta.ts`, ese número queda desactualizado el día que cambie la regla y nadie se entera.

Los límites por tipo de acción salen de la fila de `tipoAccion`; la base, el piso y el techo del puntaje salen de la fila de `institucion`. Eso ya lo estás haciendo bien.

Regla 4 — Con Convex no hay RLS: `permisos.ts` es la única defensa

Con PostgreSQL había dos capas: el código y las políticas RLS de la base. **Convex no tiene la segunda**. Si olvidas una comprobación, la consulta **no falla: devuelve los datos** — al revés de como era antes.

Ninguna lectura ni escritura sobre `estudiante`, `matricula`, `accionRegistrada`, `reporteEstudiante` o `puntajePeriodo` ocurre sin pasar antes por `permisos.ts`. Es la regla 3 del proyecto y ahora nada más la respalda.

Función	Cuándo	Devuelve
<code>exigirDocente(ctx)</code>	Quien llama tiene que ser docente	<code>Doc<"docente"></code>
<code>exigirAccesoDocenteAEstudiante(ctx, estudianteId)</code>	Ese estudiante es de un curso suyo	<code>{ docente, matricula }</code>
<code>exigirTitularDelCurso(ctx, cursoId)</code>	Es el titular vigente del curso	<code>Doc<"docente"></code>
<code>exigirRepresentante(ctx)</code>	Quien llama es representante	<code>Doc<"representante"></code>
<code>exigirVinculo(ctx, estudianteId)</code>	Ese estudiante es hijo suyo	<code>Doc<"vinculoRepresentacion"></code>

Regla 5 — Los errores tienen que llegarle a la pantalla

Un `ErrorDominio` cruza la frontera de Convex como un fallo genérico, y entonces la app solo puede decir «no pudimos completar la solicitud». Por eso `nucleo.ts` e `interaccion.ts` envuelven sus handlers. **Haz lo mismo.** Copia esta función al principio de `conducta.ts`:

```
import { ConvexError, v } from "convex/values";
import { ErrorDominio } from "../lib/guardas";
import { ErrorPermiso } from "../lib/permisos";

/** Los errores esperados conservan código y mensaje al llegar al cliente. */
async function conErroresPublicos<T>(operacion: () => Promise<T>): Promise<T> {
  try {
    return await operacion();
  } catch (error) {
    if (error instanceof ErrorDominio || error instanceof ErrorPermiso) {
      throw new ConvexError({ codigo: error.codigo, mensaje: error.mensaje });
    }
    throw error;
  }
}
```

Y cada función se declara así — fíjate que el `async` se mueve adentro:

```
export const registrarAccion = mutation({
  args: { /* ... */ },
  handler: (ctx, args) => conErroresPublicos(async () => {
    // ... tu código de siempre
  }),
});
```

Y nunca uses `throw new Error(...)`. Usa `ErrorDominio(codigo, mensaje)`, que la app sí puede distinguir de un fallo inesperado. En tu PR #39 hay tres: curso, año lectivo e institución no encontrados. Son un cambio de una línea cada uno.

Parte 5 — Cómo escribir una prueba

Lee esta parte entera antes del día 1. Es el andamio que vas a copiar veinte veces.

5.1 Crea `movil/convex/conducta.test.ts`

Este archivo no existe. Empiézalos con esto tal cual — es el sembrado completo de un docente con su curso, un estudiante matriculado y su representante:

```

// @vitest-environment edge-runtime
/// <reference types="vite/client" />

import { convexTest } from "convex-test";
import { beforeEach, describe, expect, it, vi } from "vitest";

import { api } from "../_generated/api";
import type { Id } from "../_generated/dataModel";
import schema from "../schema";

const modules = import.meta.glob(["./conducta.ts", "../_generated/*.js"]);

const AHORA = new Date("2026-09-15T15:00:00Z");

/** Un curso con su docente titular, un estudiante matriculado y su representante. */
async function sembrar(t: Return<typeof convexTest>) {
  const ids = await t.run(async (ctx) => {
    const ahora = Date.now();

    const perfilDocente = await ctx.db.insert("perfilUsuario", {
      authSubject: "https://convex.test|docente_1",
      tipoDocumento: "CEDULA", numeroDocumento: "0900000001", actualizadoEn: ahora,
    });
    const docenteId = await ctx.db.insert("docente", {
      perfilUsuarioId: perfilDocente, actualizadoEn: ahora,
    });

    const perfilRep = await ctx.db.insert("perfilUsuario", {
      authSubject: "https://convex.test|rep_1",
      tipoDocumento: "CEDULA", numeroDocumento: "0900000002", actualizadoEn: ahora,
    });
    const representanteId = await ctx.db.insert("representante", {
      perfilUsuarioId: perfilRep, actualizadoEn: ahora,
    });

    const institucionId = await ctx.db.insert("institucion", {
      nombreDeclarado: "Unidad Educativa Piloto", verificada: false,
      regimen: "COSTA_INSULAR", ciudad: "Guayaquil", zonaHoraria: "America/Guayaquil",
      puntajeBase: 60, puntajeMinimo: 0, puntajeMaximo: 100,
      topeDiarioPositivo: 4, topeDiarioNegativo: 5,
      estado: "ACTIVA", actualizadoEn: ahora,
    });
    const anioLectivoId = await ctx.db.insert("anioLectivo", {
      institucionId, nombre: "2026-2027",
      fechaInicio: "2026-05-04", fechaFin: "2027-02-26",
      estado: "EN_CURSO", actualizadoEn: ahora,
    });
    const periodoAcademicoId = await ctx.db.insert("periodoAcademico", {
      anioLectivoId, nombre: "Primer parcial", orden: 1,
      fechaInicio: "2026-05-04", fechaFin: "2026-11-10",
      estado: "EN_CURSO", actualizadoEn: ahora,
    });
    const cursoId = await ctx.db.insert("curso", {
      anioLectivoId, nombre: "Quinto A", nivel: "5to", paralelo: "A",
      jornada: "MATUTINA", estado: "ACTIVO", actualizadoEn: ahora,
    });
  });
}

```

```

await ctx.db.insert("asignacionDocente", {
  cursoId, docenteId, rol: "TITULAR",
  vigenteDesde: "2026-05-04", actualizadoEn: ahora,
});

const estudianteId = await ctx.db.insert("estudiante", {
  institucionId, nombres: "Ana", apellidos: "Perez",
  tipoDocumento: "CEDULA", numeroDocumento: "0911111111",
  origenRegistro: "REPRESENTANTE", estadoVerificacion: "APROBADO",
  estado: "ACTIVO", actualizadoEn: ahora,
});
const matriculaId = await ctx.db.insert("matricula", {
  estudianteId, cursoId, fechaIngreso: "2026-05-04",
  estado: "CURSANDO", actualizadoEn: ahora,
});
await ctx.db.insert("vinculoRepresentacion", {
  representanteId, estudianteId, parentesco: "MADRE",
  estado: "ACTIVO", vigenteDesde: "2026-05-04", actualizadoEn: ahora,
});
await ctx.db.insert("puntajePeriodo", {
  matriculaId, periodoAcademicoId,
  puntajeBase: 60, puntosPositivos: 0, puntosNegativos: 0,
  puntajeActual: 60, congelado: false, recalculadoEn: ahora,
});

// Un tipo de accion negativa y uno positivo, para tener con que probar.
const categoriaId = await ctx.db.insert("categoriaAccion", {
  institucionId, codigo: "DISCIPLINA", nombre: "Indisciplina",
  aplicaA: "ESTUDIANTE", orden: 1, activa: true, actualizadoEn: ahora,
});
const negativaId = await ctx.db.insert("tipoAccion", {
  institucionId, categoriaAccionId: categoriaId, codigo: "NEG_INDISCIPLINA",
  nombre: "Indisciplina", signo: "NEGATIVA",
  puntosDefecto: -1, puntosMin: -3, puntosMax: -1,
  requiereDescripcion: true, admiteInconformidad: true,
  cuentaEnBitacora: true, activa: true, actualizadoEn: ahora,
});
const positivaId = await ctx.db.insert("tipoAccion", {
  institucionId, categoriaAccionId: categoriaId, codigo: "POS_DESEMPEÑO",
  nombre: "Desempeno destacado", signo: "POSITIVA",
  puntosDefecto: 1, puntosMin: 1, puntosMax: 2,
  requiereDescripcion: true, admiteInconformidad: false,
  cuentaEnBitacora: true, activa: true, actualizadoEn: ahora,
});

return {
  docenteId, representanteId, estudianteId, matriculaId,
  periodoAcademicoId, institucionId, cursoId, negativaId, positivaId,
};
});

return {
  ...ids,
  docente: t.withIdentity({ subject: "docente_1" }),
  representante: t.withIdentity({ subject: "rep_1" }),
};

```

```
}  
  
beforeEach(() => vi.useFakeTimers().setSystemTime(AHORA));
```

Fíjate en el `authSubject` . Lleva el prefijo `https://convex.test|` . Es el emisor que usa `convex-test` al firmar las identidades, y desde el PR #42 de Daniel la resolución de identidad va por ahí. Si siembras el subject a secas, **tus pruebas van a fallar con «tu cuenta existe pero aún no completaste tu perfil»** y vas a perder una hora buscando el motivo. A mí me pasó.

5.2 Cómo se ve una prueba

```
describe("conducta – registrar accion", () => {  
  it("un docente de otro curso no puede registrar una accion", async () => {  
    const t = convexTest(schema, modules);  
    const e = await sembrar(t);  
  
    // Un docente ajeno, sin curso ni asignacion.  
    await t.run(async (ctx) => {  
      const perfil = await ctx.db.insert("perfilUsuario", {  
        authSubject: "https://convex.test|docente_2",  
        tipoDocumento: "CEDULA", numeroDocumento: "0900000009",  
        actualizadoEn: Date.now(),  
      });  
      await ctx.db.insert("docente", { perfilUsuarioId: perfil, actualizadoEn: Date.now() });  
    });  
  
    await expect(  
      t.withIdentity({ subject: "docente_2" }).mutation(api.conducta.registrarAccion, {  
        estudianteId: e.estudianteId, tipoAccionId: e.negativaId,  
        descripcion: "Interrumpio la clase", puntosAplicados: -1,  
      }),  
    ).rejects.toThrow("no pertenece a un curso tuyo");  
  });  
});
```

5.3 Correr las pruebas

```
npm test          # todas, desde la raiz del repositorio  
npm test -- conducta  # solo las tuyas, mientras trabajas
```

`convex-test` corre en memoria: **no toca tu despliegue** y no necesita internet. Cada `convexTest(schema, modules)` arranca con la base vacía.

Parte 6 — Día 1: cerrar el issue #9

DÍA 1

Media jornada de trabajo. Al final de hoy tu PR #39 queda aprobado.

6.1 Las tres pruebas que bloquean la aprobación

No es criterio mío: está en la definición de terminado (`backlog-y-reparto.md` §6) y en la plantilla del PR, que quedó sin marcar.

Prueba 1 — un docente que no es del curso recibe `ErrorPermiso`

Es la de §5.2, cópiala tal cual. **Esta es la importante.** Sin RLS debajo, si un día alguien mueve una línea y se salta `exigirAccesoDocenteAEstudiente`, la consulta no falla: devuelve los datos de un menor a quien no le toca. Esta prueba es lo único que avisaría.

Prueba 2 — pasarse del tope diario rechaza la acción

```
it("no deja pasar el tope diario de puntos negativos", async () => {
  const t = convexTest(schema, modules);
  const e = await sembrar(t);
  const accion = (puntos: number) =>
    e.docente.mutation(api.conducta.registrarAccion, {
      estudianteId: e.estudianteId, tipoAccionId: e.negativaId,
      descripcion: "Interrumpio la clase", puntosAplicados: puntos,
    });

  await accion(-3);           // acumulado: -3
  await accion(-2);           // acumulado: -5, justo en el tope
  await expect(accion(-1)).rejects.toThrow("tope"); // -6 se pasa

  const acciones = await t.run((ctx) => ctx.db.query("accionRegistrada").collect());
  expect(acciones).toHaveLength(2); // la tercera no se escribio
});
```

Fíjate en la última afirmación: no basta con que lance el error, hay que comprobar que **no escribió nada**. Un error después de insertar deja la base sucia.

Prueba 3 — el recálculo da el número correcto

```
it("el puntaje se recalcula desde las acciones vigentes", async () => {
  const t = convexTest(schema, modules);
  const e = await sembrar(t);

  await e.docente.mutation(api.conducta.registrarAccion, {
    estudianteId: e.estudianteId, tipoAccionId: e.negativaId,
    descripcion: "Interrumpio la clase", puntosAplicados: -3,
  });
  await e.docente.mutation(api.conducta.registrarAccion, {
    estudianteId: e.estudianteId, tipoAccionId: e.positivaId,
    descripcion: "Ayudo a un companero", puntosAplicados: 2,
  });

  const puntaje = await t.run(async (ctx) =>
    await ctx.db.query("puntajePeriodo")
      .withIndex("por_matricula_periodo", (q) =>
        q.eq("matriculaId", e.matriculaId).eq("periodoAcademicoId", e.periodoAcademicoId))
      .unique(),
  );
  expect(puntaje?.puntajeActual).toBe(59); // 60 - 3 + 2
});
```

6.2 anularAccion

El issue #9 la pide junto con `registrarAccion`. Reglas:

- Solo sobre acciones **negativas**. Una positiva no se anula — no hay nada que reparar.
- Pasa a estado `ANULADA` y **vale 0 puntos** en el recálculo.
- Se **recalcula** el puntaje. Nunca se le suman de vuelta los puntos.
- Nada se borra. La acción sigue en la bitácora, marcada.

```

export const anularAccion = mutation({
  args: {
    accionRegistradaId: v.id("accionRegistrada"),
    motivo: v.string(),
  },
  handler: (ctx, args) => conErroresPublicos(async () => {
    const accion = await ctx.db.get(args.accionRegistradaId);
    if (accion === null) throw new ErrorDominio("NO_ENCONTRADO", "Esa anotacion no existe.");

    const matricula = await ctx.db.get(accion.matriculaId);
    if (matricula === null) throw new ErrorDominio("NO_ENCONTRADO", "La matricula no existe.");

    // El permiso se comprueba contra el estudiante dueño de la accion.
    const { docente } = await exigirAccesoDocenteAEstudiante(ctx, matricula.estudianteId);

    if (accion.signo !== "NEGATIVA") {
      throw new ErrorDominio("VALIDACION", "Solo se pueden anular las anotaciones negativas.");
    }
    if (accion.estado !== "VIGENTE") {
      throw new ErrorDominio("CONFLICTO", "Esa anotacion ya fue anulada o modificada.");
    }

    const ahora = Date.now();
    await ctx.db.patch(accion._id, {
      estado: "ANULADA",
      resueltaPorDocenteId: docente._id,
      resueltaEn: ahora,
      motivoResolucion: args.motivo.trim(),
      actualizadoEn: ahora,
    });

    await recalcularPuntaje(ctx, matricula._id, accion.periodoAcademicoId);
    return accion._id;
  }),
});

```

6.3 Saca el recálculo a su propia función

Ahora lo tienes dentro de `registrarAccion`. Sácalo: lo van a llamar `registrarAccion`, `anularAccion`, el cron nocturno y **Kenny desde `interaccion.ts`** cuando un reclamo se resuelva como modificada.

```

/**
 * ADR-005: el puntaje se recalcula desde las acciones VIGENTES, nunca se suma.
 * Respetar `congelado`: si el parcial ya cerro, no se toca.
 */
export async function recalcularpuntaje(
  ctx: MutationCtx,
  matriculaId: Id<"matricula">,
  periodoAcademicoId: Id<"periodoAcademico">,
): Promise<number | null> {
  const fila = await ctx.db.query("puntajePeriodo")
    .withIndex("por_matricula_periodo", (q) =>
      q.eq("matriculaId", matriculaId).eq("periodoAcademicoId", periodoAcademicoId))
    .unique();
  if (fila === null || fila.congelado) return fila?.puntajeActual ?? null;

  const vigentes = await ctx.db.query("accionRegistrada")
    .withIndex("por_matricula_periodo", (q) =>
      q.eq("matriculaId", matriculaId)
        .eq("periodoAcademicoId", periodoAcademicoId)
        .eq("estado", "VIGENTE"))
    .collect();

  const puntos = vigentes.map((a) => a.puntosAplicados);
  const puntajeActual = calcularPuntaje(
    puntos, fila.puntajeBase, REGLAS.PUNTAJE_MINIMO, REGLAS.PUNTAJE_MAXIMO,
  );

  await ctx.db.patch(fila._id, {
    puntosPositivos: puntos.filter((p) => p > 0).reduce((a, b) => a + b, 0),
    puntosNegativos: puntos.filter((p) => p < 0).reduce((a, b) => a + b, 0),
    puntajeActual,
    franjaConductaId: await franjaDe(ctx, puntajeActual),
    recalculadoEn: Date.now(),
  });
  return puntajeActual;
}

```

Y la franja, que vas a necesitar también en los reportes:

```

/** La franja de conducta cuyo rango contiene el puntaje. */
async function franjaDe(ctx: QueryCtx, puntaje: number) {
  const franjas = await ctx.db.query("franjaConducta").collect();
  return franjas.find((f) => puntaje >= f.puntajeDesde && puntaje <= f.puntajeHasta)?._id;
}

```

6.4 La auditoría que Kenny está esperando

DP-006 fija cuatro eventos para la v1. Tres ya tienen quien los escriba. **El cuarto es tuyo**, y son dos llamadas:

```
// Al final de registrarAccion, despues de insertar y recalcular:
await auditar(ctx, {
  accion: "CREAR",
  entidadTipo: "accionRegistrada",
  entidadId: idAccion,
  institucionId: institucion._id,
  datosDespues: {
    matriculaId: matricula._id,
    tipoAccionId: tipo._id,
    puntosAplicados: args.puntosAplicados,
    fechaOcurrancia: fecha,
  },
});

// En anularAccion:
await auditar(ctx, {
  accion: "ANULAR",
  entidadTipo: "accionRegistrada",
  entidadId: accion._id,
  institucionId: institucion._id,
  datosAntes: { estado: "VIGENTE", puntosAplicados: accion.puntosAplicados },
  datosDespues: { estado: "ANULADA", motivo: args.motivo },
});
```

`institucionId` es opcional, pero ponlo: es lo que permite responderle a **un colegio concreto** qué pasó con sus estudiantes. Sale de `matricula` → `curso` → `anioLectivo` → `institucionId`, que ya recorres.

6.5 Cierra el día

```
npm run typecheck && npm test
```

Las dos en verde, y actualiza el PR #39:

- Marca las casillas de la plantilla que ahora sí se cumplen.
- Escribe `Cierra #9` en el cuerpo del PR.
- Comenta pidiendo revisión.

No toques `.env.schema`. El arreglo del UUID que hiciste es idéntico al del PR #27 ya fusionado, así que no va a dar conflicto — pero tampoco hace falta que lo mantengas.

Parte 7 — Día 2: asistencia

DÍA 2

7.1 Lo que hay que resolver

El docente toma asistencia del curso desde el celular. Estados en `ESTADO_ASISTENCIA` : `PRESENTE` , `AUSENTE` , `ATRASO` , `JUSTIFICADA` , `PERMISO` .

Una sola marca por estudiante por día. Convex **no tiene índice único**. Esa unicidad la compruebas tú, con una lectura indexada **dentro de la misma mutation**, antes de insertar. Como la mutation es serializable, eso alcanza.

7.2 La mutation

Recibe el curso y la lista completa, no estudiante por estudiante: el docente marca a los 30 y guarda una vez.

```

export const tomarAsistencia = mutation({
  args: {
    cursoId: v.id("curso"),
    fecha: v.optional(v.string()),
    marcas: v.array(v.object({
      estudianteId: v.id("estudiante"),
      estado: v.union(...ESTADO_ASISTENCIA.map((e) => v.literal(e))),
      observacion: v.optional(v.string()),
    })),
  },
  handler: (ctx, args) => conErroresPublicos(async () => {
    const docente = await exigirTitularDelCurso(ctx, args.cursoId);
    const fecha = args.fecha ?? hoyEnGuayaquil();
    if (fecha > hoyEnGuayaquil()) {
      throw new ErrorDominio("FECHAS_INVALIDAS", "No se puede tomar asistencia de un día futuro.");
    }

    const periodo = await periodoEnCurso(ctx, args.cursoId);
    const ahora = Date.now();
    let creadas = 0;
    let actualizadas = 0;

    for (const marca of args.marcas) {
      // El permiso se comprueba estudiante por estudiante: nadie cuela
      // en la lista a un alumno de otro curso.
      const { matricula } = await exigirAccesoDocenteAEstudiante(ctx, marca.estudianteId);
      if (matricula.cursoId !== args.cursoId) {
        throw new ErrorDominio("VALIDACION", "Ese estudiante no pertenece a este curso.");
      }

      const previa = await ctx.db.query("registroAsistencia")
        .withIndex("por_matricula_fecha", (q) =>
          q.eq("matriculaId", matricula._id).eq("fecha", fecha))
        .unique();

      const datos = {
        estado: marca.estado,
        observacion: marca.observacion?.trim() || undefined,
        registradoPorDocenteId: docente._id,
        actualizadoEn: ahora,
      };

      if (previa === null) {
        await ctx.db.insert("registroAsistencia", {
          matriculaId: matricula._id,
          periodoAcademicoId: periodo._id,
          fecha, ...datos,
        });
        creadas++;
      } else {
        // Corregir una marca del mismo día es normal: el docente se equivoca.
        await ctx.db.patch(previa._id, datos);
        actualizadas++;
      }
    }
  })
});
return { creadas, actualizadas, fecha };

```

```
  }},  
  });
```

7.3 La consulta para la pantalla

El docente abre la pantalla y tiene que ver a los 30 con lo que ya marcó hoy, si marcó:

```
export const asistenciaDelDia = query({  
  args: { cursoId: v.id("curso"), fecha: v.optional(v.string()) },  
  handler: (ctx, args) => conErroresPublicos(async () => {  
    await exigirTitularDelCurso(ctx, args.cursoId);  
    const fecha = args.fecha ?? hoyEnGuayaquil();  
  
    const matriculas = await ctx.db.query("matricula")  
      .withIndex("por_curso_estado", (q) =>  
        q.eq("cursoId", args.cursoId).eq("estado", "CURSANDO"))  
      .collect();  
  
    const filas = [];  
    for (const matricula of matriculas) {  
      const estudiante = await ctx.db.get(matricula.estudianteId);  
      if (estudiante === null) continue;  
      const marca = await ctx.db.query("registroAsistencia")  
        .withIndex("por_matricula_fecha", (q) =>  
          q.eq("matriculaId", matricula._id).eq("fecha", fecha))  
        .unique();  
      filas.push({  
        estudianteId: estudiante._id,  
        nombres: estudiante.nombres,  
        apellidos: estudiante.apellidos,  
        estado: marca?.estado ?? null,  
        observacion: marca?.observacion ?? null,  
      });  
    }  
    return { fecha, estudiantes: filas.sort((a, b) => a.apellidos.localeCompare(b.apellidos)) };  
  }),  
});
```

7.4 Pruebas del día 2

- ☐ Tomar asistencia dos veces el mismo día **corrige**, no duplica: al final hay una sola fila por estudiante.
- ☐ Un docente que no es titular del curso recibe `ErrorPermiso`.
- ☐ Un estudiante de otro curso dentro de la lista hace fallar la mutation entera.
- ☐ `asistenciaDelDia` devuelve a todos los `CURSANDO`, con `estado: null` los que no tienen marca.

Parte 8 — Días 3 y 4: el reporte del día

DÍAS 3-4

Es lo que el representante abre cada mañana. La razón por la que la app existe.

8.1 Los cuatro campos salen de la base, no del código

DP-004: cuatro campos, **ninguno obligatorio**. Vienen sembrados en `plantillaCampo`:

Código	Etiqueta	Tipo	Orden
ANUNCIOS	Anuncios	TEXTO_LARGO	1
NOVEDADES	Novedades del día	TEXTO_LARGO	2
TAREAS	Tareas enviadas	TEXTO_LARGO	3
CONSEJO	Consejo del día	TEXTO_CORTO	4

Léelos de la base de datos. Si los escribes fijos en el código, el día que una institución quiera un quinto campo hay que recompilar la app.

```
export const camposDelReporte = query({
  args: {},
  handler: (ctx) => conErroresPublicos(async () => {
    await exigirDocente(ctx);
    const plantilla = await ctx.db.query("plantillaReporte")
      .withIndex("por_institucion", (q) => q.eq("institucionId", undefined).eq("activa", true))
      .first();
    if (plantilla === null) throw new ErrorDominio("NO_ENCONTRADO", "No hay plantilla de reporte.");

    const campos = await ctx.db.query("plantillaCampo")
      .withIndex("por_plantilla_codigo", (q) => q.eq("plantillaReporteId", plantilla._id))
      .collect();
    return {
      plantillaReporteId: plantilla._id,
      campos: campos.filter((c) => c.activo).sort((a, b) => a.orden - b.orden)
        .map((c) => ({
          id: c._id, codigo: c.codigo, etiqueta: c.etiqueta,
          tipoDato: c.tipoDato, textoAyuda: c.textoAyuda ?? null,
          longitudMaxima: c.longitudMaxima ?? null,
        })),
    };
  }),
});
```

8.2 Guardar el borrador

El docente escribe durante el día y guarda varias veces. Mientras esté en `BORRADOR` se puede editar.

```

export const guardarReporteGeneral = mutation({
  args: {
    cursoId: v.id("curso"),
    fecha: v.optional(v.string()),
    valores: v.array(v.object({
      plantillaCampoId: v.id("plantillaCampo"),
      valorTexto: v.string(),
    })),
  },
  handler: (ctx, args) => conErroresPublicos(async () => {
    await exigirTitularDelCurso(ctx, args.cursoId);
    const fecha = args.fecha ?? hoyEnGuayaquil();
    const periodo = await periodoEnCurso(ctx, args.cursoId);

    // ux_reporte_general: uno por curso y dia. Unicidad comprobada aqui.
    let reporte = await ctx.db.query("reporteGeneral")
      .withIndex("por_curso_fecha", (q) => q.eq("cursoId", args.cursoId).eq("fecha", fecha))
      .unique();

    if (reporte !== null && reporte.estado !== "BORRADOR") {
      throw new ErrorDominio("CONFLICTO", "El reporte de ese dia ya fue publicado.");
    }

    if (reporte === null) {
      const plantilla = await plantillaActiva(ctx);
      const id = await ctx.db.insert("reporteGeneral", {
        cursoId: args.cursoId, periodoAcademicoId: periodo._id,
        plantillaReporteId: plantilla._id, fecha,
        estado: "BORRADOR", actualizadoEn: Date.now(),
      });
      reporte = (await ctx.db.get(id))!;
    }

    for (const valor of args.valores) {
      const previo = await ctx.db.query("reporteGeneralValor")
        .withIndex("por_reporte_campo", (q) =>
          q.eq("reporteGeneralId", reporte!._id).eq("plantillaCampoId", valor.plantillaCampoId))
        .unique();
      const texto = valor.valorTexto.trim() || undefined;
      if (previo === null) {
        await ctx.db.insert("reporteGeneralValor", {
          reporteGeneralId: reporte._id,
          plantillaCampoId: valor.plantillaCampoId,
          valorTexto: texto,
        });
      } else {
        await ctx.db.patch(previo._id, { valorTexto: texto });
      }
    }
    return reporte._id;
  }),
});

```

8.3 Publicar: la parte que importa

Al publicar se genera un **reporteEstudiante** por cada estudiante con matrícula **CURSANDO**. Ese reporte es una **fotografía inmutable del día**: una vez publicado no cambia, aunque después se corrija una acción.

Si no hubo reporte general pero sí hubo acciones, igual se genera el del estudiante. Un padre cuyo hijo tuvo una anotación no puede quedarse sin ver nada porque el docente no llenó los cuatro campos.

```

export const publicarReporteGeneral = mutation({
  args: { cursoId: v.id("curso"), fecha: v.optional(v.string()) },
  handler: (ctx, args) => conErroresPublicos(async () => {
    const docente = await exigirTitularDelCurso(ctx, args.cursoId);
    const fecha = args.fecha ?? hoyEnGuayaquil();
    const periodo = await periodoEnCurso(ctx, args.cursoId);
    const ahora = Date.now();

    const reporte = await ctx.db.query("reporteGeneral")
      .withIndex("por_curso_fecha", (q) => q.eq("cursoId", args.cursoId).eq("fecha", fecha))
      .unique();
    if (reporte !== null && reporte.estado === "PUBLICADO") {
      throw new ErrorDominio("CONFLICTO", "Ese reporte ya fue publicado.");
    }
    if (reporte !== null) {
      await ctx.db.patch(reporte._id, {
        estado: "PUBLICADO", publicadoEn: ahora,
        publicadoPorDocenteId: docente._id, actualizadoEn: ahora,
      });
    }

    const matriculas = await ctx.db.query("matricula")
      .withIndex("por_curso_estado", (q) =>
        q.eq("cursoId", args.cursoId).eq("estado", "CURSANDO"))
      .collect();

    let generados = 0;
    for (const matricula of matriculas) {
      // Idempotente: republicar no duplica el reporte de nadie.
      const previo = await ctx.db.query("reporteEstudiante")
        .withIndex("por_matricula_fecha", (q) =>
          q.eq("matriculaId", matricula._id).eq("fecha", fecha))
        .unique();
      if (previo !== null) continue;

      const acciones = await ctx.db.query("accionRegistrada")
        .withIndex("por_matricula_fecha", (q) =>
          q.eq("matriculaId", matricula._id).eq("fechaOcurriencia", fecha))
        .collect();
      const vigentes = acciones.filter((a) => a.estado === "VIGENTE");

      const asistencia = await ctx.db.query("registroAsistencia")
        .withIndex("por_matricula_fecha", (q) =>
          q.eq("matriculaId", matricula._id).eq("fecha", fecha))
        .unique();

      const puntaje = await ctx.db.query("puntajePeriodo")
        .withIndex("por_matricula_periodo", (q) =>
          q.eq("matriculaId", matricula._id).eq("periodoAcademicoId", periodo._id))
        .unique();
      const puntajeAlCierre = puntaje?.puntajeActual ?? REGLAS.PUNTAJE_BASE;

      const reporteEstudianteId = await ctx.db.insert("reporteEstudiante", {
        matriculaId: matricula._id,
        periodoAcademicoId: periodo._id,
        reporteGeneralId: reporte?._id,
      });
    }
  })
});

```

```

    fecha,
    tieneNovedades: vigentes.length > 0,
    puntajeAlCierre,
    franjaConductaId: await franjaDe(ctx, puntajeAlCierre),
    estadoAsistencia: asistencia?.estado,
    generadoEn: ahora,
  });
  generados++;

  let orden = 0;
  for (const accion of vigentes) {
    await ctx.db.insert("reporteEstudianteItem", {
      reporteEstudianteId, tipoItem: "ACCION",
      accionRegistradaId: accion._id, orden: orden++,
    });
  }
  if (asistencia !== null) {
    await ctx.db.insert("reporteEstudianteItem", {
      reporteEstudianteId, tipoItem: "ASISTENCIA",
      registroAsistenciaId: asistencia._id, orden: orden++,
    });
  }

  // Una entrega por cada representante vinculado.
  const vinculo = await ctx.db.query("vinculoRepresentacion")
    .withIndex("por_estudiante_estado", (q) =>
      q.eq("estudianteId", matricula.estudianteId).eq("estado", "ACTIVO"))
    .unique();
  if (vinculo !== null) {
    await ctx.db.insert("entregaReporte", {
      reporteEstudianteId, representanteId: vinculo.representanteId,
      entregadoEn: ahora,
    });
  }
}
return { fecha, generados };
}),
});
});

```

8.4 Pruebas de los días 3 y 4

- ☐ Los campos salen de `plantillaCampo` y son los cuatro sembrados.
- ☐ Guardar dos veces el mismo día actualiza el borrador, no crea un segundo.
- ☐ Publicar genera un `reporteEstudiante` por cada matrícula `CURSANDO`, y ninguno para las que no lo están.
- ☐ Un reporte publicado ya no se puede editar: `guardarReporteGeneral` lanza `CONFLICTO`.
- ☐ Publicar dos veces no duplica reportes de estudiante.
- ☐ Un estudiante con una acción del día tiene `tieneNovedades: true` y su ítem `ACCION`.
- ☐ Sin reporte general, un estudiante con acciones **igual** recibe su reporte.
- ☐ Un docente que no es titular recibe `ErrorPermiso`.

Parte 9 — Día 5: las tres consultas del representante

DÍA 5

Este es el día que más destraba al equipo. Con estas tres consultas se pueden construir P4, P5 y P6 — y P4 es la pantalla más importante del producto. Si vas retrasada, haz este día antes que el 3 y el 4.

En las tres: `exigirVinculo(ctx, estudianteId)` **antes** de leer nada. Un representante solo ve a sus propios hijos. Sin RLS debajo, olvidarlo no da error: entrega los datos de un menor ajeno.

9.1 El reporte de hoy (P4)

Tiene que poder distinguir «**no hay**» de «**error**»: la app muestra «aún no tienes reporte nuevo», que no es lo mismo que una pantalla rota.

```
export const reporteDeHoy = query({
  args: { estudianteId: v.id("estudiante"), fecha: v.optional(v.string()) },
  handler: (ctx, args) => conErroresPublicos(async () => {
    await exigirVinculo(ctx, args.estudianteId);
    const fecha = args.fecha ?? hoyEnGuayaquil();

    const matricula = await ctx.db.query("matricula")
      .withIndex("por_estudiante_estado", (q) =>
        q.eq("estudianteId", args.estudianteId).eq("estado", "CURSANDO"))
      .unique();
    if (matricula === null) return { fecha, hay: false as const, reporte: null };

    const reporte = await ctx.db.query("reporteEstudiante")
      .withIndex("por_matricula_fecha", (q) =>
        q.eq("matriculaId", matricula._id).eq("fecha", fecha))
      .unique();
    if (reporte === null) return { fecha, hay: false as const, reporte: null };

    return { fecha, hay: true as const, reporte: await presentarReporte(ctx, reporte) };
  }),
});
```

Y el presentador, que vas a reutilizar en las tres:

```

async function presentarReporte(ctx: QueryCtx, reporte: Doc<"reporteEstudiante">) {
  const items = await ctx.db.query("reporteEstudianteItem")
    .withIndex("por_reporte", (q) => q.eq("reporteEstudianteId", reporte._id))
    .collect();

  const acciones = [];
  for (const item of items) {
    if (item.tipoItem !== "ACCION" || item.accionRegistradaId === undefined) continue;
    const accion = await ctx.db.get(item.accionRegistradaId);
    if (accion === null) continue;
    const categoria = await ctx.db.get(accion.categoriaAccionId);
    acciones.push({
      id: accion._id,
      categoria: categoria?.nombre ?? "",
      signo: accion.signo,
      puntos: accion.puntosAplicados,
      descripcion: accion.descripcion,
      estado: accion.estado,
    });
  }

  const franja = reporte.franjaConductaId
    ? await ctx.db.get(reporte.franjaConductaId) : null;

  // Los campos del reporte general del curso, si lo hubo.
  const generales = [];
  if (reporte.reporteGeneralId) {
    const valores = await ctx.db.query("reporteGeneralValor")
      .withIndex("por_reporte_campo", (q) =>
        q.eq("reporteGeneralId", reporte.reporteGeneralId!))
      .collect();
    for (const valor of valores) {
      if (!valor.valorTexto) continue;
      const campo = await ctx.db.get(valor.plantillaCampoId);
      if (campo) generales.push({ etiqueta: campo.etiqueta, texto: valor.valorTexto, orden:
campo.orden });
    }
    generales.sort((a, b) => a.orden - b.orden);
  }

  return {
    id: reporte._id,
    fecha: reporte.fecha,
    tieneNovedades: reporte.tieneNovedades,
    puntaje: reporte.puntajeAlCierre,
    franja: franja ? { nombre: franja.nombre, frase: franja.fraseRepresentante, color:
franja.colorHex ?? null } : null,
    asistencia: reporte.estadoAsistencia ?? null,
    acciones,
    general: generales.map(({ etiqueta, texto }) => ({ etiqueta, texto })),
  };
}

```

9.2 Reportes anteriores, limitados por plan (P5)

El plan gratuito ve menos que el premium. **Los dos números salen de REGLAS :**
`REPORTES_PREVIOS_FREE` (2) y `REPORTES_PREVIOS_PREMIUM` (7). No los escribas a mano.

```
import { tieneAccesoVigente } from "../lib/revenuecat";

export const reportesAnteriores = query({
  args: { estudianteId: v.id("estudiante") },
  handler: (ctx, args) => conErroresPublicos(async () => {
    const vinculo = await exigirVinculo(ctx, args.estudianteId);
    const representante = await ctx.db.get(vinculo.representanteId);

    const premium = representante !== null &&
      (await tieneAccesoVigente(ctx, representante.perfilUsuarioId, ENTITLEMENTS.REPRESENTANTE));
    const limite = premium ? REGLAS.REPORTES_PREVIOS_PREMIUM : REGLAS.REPORTES_PREVIOS_FREE;

    const matricula = await ctx.db.query("matricula")
      .withIndex("por_estudiante_estado", (q) =>
        q.eq("estudianteId", args.estudianteId).eq("estado", "CURSANDO"))
      .unique();
    if (matricula === null) return { limite, premium, reportes: [] };

    // El indice esta ordenado por fecha: `order("desc")` da los mas recientes.
    const reportes = await ctx.db.query("reporteEstudiante")
      .withIndex("por_matricula_fecha", (q) => q.eq("matriculaId", matricula._id))
      .order("desc")
      .take(limite + 1); // +1 = el de hoy, que no cuenta como "anterior"

    const hoy = hoyEnGuayaquil();
    const anteriores = reportes.filter((r) => r.fecha < hoy).slice(0, limite);
    return {
      limite, premium,
      reportes: await Promise.all(anteriores.map((r) => presentarReporte(ctx, r))),
    };
  }),
});
```

Devuelve `limite` y `premium` en la respuesta. La pantalla los necesita para decir «con Premium verías los últimos 7» sin tener que adivinar el plan por su cuenta.

9.3 El acumulado del parcial (P6)

La bitácora de acciones del período, el puntaje y su franja.

```

export const reporteAcumulado = query({
  args: { estudianteId: v.id("estudiante") },
  handler: (ctx, args) => conErroresPublicos(async () => {
    await exigirVinculo(ctx, args.estudianteId);

    const matricula = await ctx.db.query("matricula")
      .withIndex("por_estudiante_estado", (q) =>
        q.eq("estudianteId", args.estudianteId).eq("estado", "CURSANDO"))
      .unique();
    if (matricula === null) throw new ErrorDominio("NO_ENCONTRADO", "El estudiante no tiene matricula
vigente.");

    const curso = await ctx.db.get(matricula.cursoId);
    const periodo = curso === null ? null : await periodoEnCursoDeCurso(ctx, curso);
    if (periodo === null) throw new ErrorDominio("PERIODO_NO_VIGENTE", "No hay un parcial en
curso.");

    const puntaje = await ctx.db.query("puntajePeriodo")
      .withIndex("por_matricula_periodo", (q) =>
        q.eq("matriculaId", matricula._id).eq("periodoAcademicoId", periodo._id))
      .unique();

    // Toda la bitacora, incluidas anuladas y modificadas: el representante
    // tiene derecho a ver que una anotacion se corrigio, no que desaparecio.
    const acciones = await ctx.db.query("accionRegistrada")
      .withIndex("por_matricula_periodo", (q) =>
        q.eq("matriculaId", matricula._id).eq("periodoAcademicoId", periodo._id))
      .collect();

    const franja = puntaje?.franjaConductaId
      ? await ctx.db.get(puntaje.franjaConductaId) : null;

    return {
      periodo: { nombre: periodo.nombre, fechaInicio: periodo.fechaInicio, fechaFin: periodo.fechaFin
    },
      puntaje: puntaje?.puntajeActual ?? REGLAS.PUNTAJE_BASE,
      puntosPositivos: puntaje?.puntosPositivos ?? 0,
      puntosNegativos: puntaje?.puntosNegativos ?? 0,
      congelado: puntaje?.congelado ?? false,
      franja: franja ? { nombre: franja.nombre, frase: franja.fraseRepresentante, color:
franja.colorHex ?? null } : null,
      bitacora: acciones
        .sort((a, b) => b.fechaOcurrencia.localeCompare(a.fechaOcurrencia))
        .map((a) => ({
          id: a._id, fecha: a.fechaOcurrencia, signo: a.signo,
          puntos: a.estado === "VIGENTE" ? a.puntosAplicados : 0,
          descripcion: a.descripcion, estado: a.estado,
        })),
    };
  }
});

```

9.4 Pruebas del día 5

- ☐ Un representante **sin vínculo** recibe `ErrorPermiso` en las tres consultas. **Es la prueba S-1 de la matriz de permisos** — «el Padre A no puede ver la bitácora del Hijo B».
- ☐ Sin reporte de hoy, `reporteDeHoy` devuelve `hay: false` y no lanza error.
- ☐ Un representante gratuito ve 2 reportes anteriores; uno premium ve 7.
- ☐ El de hoy no aparece dentro de «anteriores».
- ☐ La bitácora acumulada incluye las anuladas, y sus puntos salen en 0.

Parte 10 — Día 6: comunicados y cron nocturno

DÍA 6

10.1 Comunicados del curso (DP-003)

Dos tipos, y **ninguno afecta el puntaje**:

Tipo	Qué es	Visibilidad
NOTA_PROFESOR	Mensaje libre, a un representante o a todo el curso	Entre 1 y 7 días, el docente elige
EVENTO	Texto largo con fecha	Hasta la fecha del evento; desaparece solo ese día

Guardas que ya existen — úsalas, no reescribas la validación:

Guarda	Qué comprueba
<code>exigirVentanaComunicado(desde, hasta)</code>	Que la ventana de visibilidad sea coherente
<code>exigirDuracionNota(dias)</code>	Entre <code>NOTA_PROFESOR_DIAS_MIN</code> y <code>MAX</code> (1 y 7)
<code>exigirFechaEvento(tipo, fechaEvento)</code>	Que un evento traiga fecha y una nota no
<code>exigirAlcanceCoherente(alcance, estudianteId)</code>	Alcance <code>CURSO</code> no lleva estudiante; <code>ESTUDIANTE</code> sí
<code>exigirPublicacionCompleta(...)</code>	Que publicar traiga fecha y autor

Y `sumarDias(fecha, dias)` para calcular `visibleHasta` sin salir de la zona horaria del piloto.

10.2 El cron nocturno

Convex tiene cron propio. **El archivo ya existe:** `movil/convex/crons.ts`, con el cron de Kenny dentro. Convex exige que todas las tareas vivan en ese único archivo, así que es el segundo sitio del proyecto donde dos personas escriben en el mismo archivo.

La regla para no pisarse: añade tu bloque **debajo** del último, nunca edites el de otro, y pon el módulo por delante del nombre. Te dejé el comentario en el sitio exacto.

```
// En movil/convex/crons.ts, debajo del cron de interaccion:

/**
 * Conducta (Persona B) – cierra el dia: publica los reportes que quedaron en
 * borrador y deja el puntaje al dia. 22:00 en Guayaquil = 03:00 UTC.
 */
crons.daily(
  "conducta: cierre nocturno",
  { hourUTC: 3, minuteUTC: 0 },
  internal.conducta.cierreNocturno,
  {},
);
```

Y en `conducta.ts`, una `internalMutation` — interna a propósito: la dispara el cron, nunca el cliente.

```
export const cierreNocturno = internalMutation({
  args: { fecha: v.optional(v.string()) },
  handler: async (ctx, args) => {
    const fecha = args.fecha ?? hoyEnGuayaquil();
    // 1. Cursos activos con reporte en BORRADOR de esa fecha -> publicar
    // 2. Estudiantes con acciones del dia y sin reporteEstudiante -> generarlo
    // 3. Devolver un resumen: { publicados, generados }
    // Acota el lote (take) para no pasarte de los limites de una transaccion.
  },
});
```

Fíjate en las horas: Guayaquil es UTC−5 todo el año, así que hora local + 5 = hora UTC. El cron de Kenny corre a las 08:00 UTC (03:00 local); el tuyo a las 03:00 UTC (22:00 local del día que cierra).

10.3 Pruebas del día 6

- ☐ Una nota de más de 7 días se rechaza.
- ☐ Un evento sin fecha se rechaza; una nota con fecha de evento también.
- ☐ Una nota de alcance `ESTUDIANTE` sin estudiante se rechaza.
- ☐ El cierre nocturno publica un borrador pendiente y no toca los ya publicados.
- ☐ Correr el cierre dos veces no duplica reportes.

Parte 11 — Día 7: lo que los demás necesitan de ti

DÍA 7

11.1 El recálculo para Kenny

En `interaccion.ts` hay un comentario esperándote:

```
// RECALCULO_PENDIENTE (#9, Persona B): cuando `conducta.ts` exponga el
// recalculation del periodo, se invoca aqui con accion.matriculaId y
// accion.periodoAcademicoId.
```

Es el caso F4: cuando un docente resuelve un reclamo como `RESUELTA_MODIFICADA`, la acción pasa a `MODIFICADA`, vale 0 puntos y **hay que devolverle los puntos al estudiante**. Si tu `recalcularPuntaje` está exportada (§6.3), Kenny solo tiene que importarla y llamarla.

Avísale en el grupo cuando esté disponible, con el nombre exacto y sus argumentos. Es un cambio de dos líneas de su lado.

11.2 Avisa a la interfaz

Escribe un comentario en los issues **#31** y **#32** listando las funciones que expusiste, con sus argumentos y qué devuelven. Quien construya esas pantallas no debería tener que leer tu código para saber cómo llamarte.

Mira `docs/04-guias/contrato-nucleo.md` como modelo: es exactamente eso para el módulo de Daniel.

11.3 Repaso final

- ☐ `npm run typecheck` y `npm test` en verde.
- ☐ Ninguna función pública sin su llamada a `permisos.ts` en la primera línea.
- ☐ Ningún número suelto: todo sale de `REGLAS`, de `tipoAccion` o de `institucion`.
- ☐ Ningún `throw new Error(...)`: todos son `ErrorDominio`.
- ☐ Ningún `puntaje = puntaje ± n`: todo pasa por `recalcularPuntaje`.
- ☐ Los issues #8, #9 y #10 cerrados o con su PR marcando `Cierra #N`.

Parte 12 — Cómo se entrega

12.1 El ciclo completo

```
# 1. Rama nueva desde main al dia
git checkout main && git pull
git checkout -b feat/conducta-asistencia

# 2. Trabajas, y compruebas antes de cada commit
npm run typecheck && npm test

# 3. Commit con mensaje que explique el porque, no el que
git add -A
git commit -m "feat(conducta): asistencia del curso con unicidad por dia"

# 4. Subir y abrir el PR
git push -u origin feat/conducta-asistencia
gh pr create --base main      # o desde la web
```

12.2 La plantilla del PR

Se rellena sola al abrirlo. **Marca las casillas de verdad**, no por costumbre. La sección «Si toca datos de estudiantes» aplica a casi todo lo tuyo: tus tablas son las cuatro más sensibles del sistema.

12.3 El CI

Cada PR corre `npm ci`, `npm run typecheck` y `npm test` en una máquina limpia con Node 24. Si pasa en tu máquina y falla ahí, casi siempre es un archivo que no subiste.

12.4 Un conflicto que te va a pasar

`movil/convex/_generated/api.d.ts` está versionado pero es **generado**. Dos ramas que añaden funciones chocan siempre ahí. **No lo resuelvas a mano:**

```
cd movil && npx convex codegen && git add convex/_generated/api.d.ts
```

12.5 Si te trabas

Escribe en el grupo con: qué intentabas, qué esperabas, qué pasó y el mensaje de error completo. Media hora atascada en silencio cuesta más que un mensaje.

Y si `convex-test` se te resiste, dímelo: es la primera vez que escribes pruebas con esa herramienta y tiene su truco. Lo vemos juntos y no pasa nada.

Apéndices

Apéndice A — Tus tablas, campo por campo

accionRegistrada

matriculaId · periodoAcademicoId · tipoAccionId · categoriaAccionId · signo · puntosAplicados · cuentaEnBitacora · descripcion · fechaOcurrancia · registradaPorDocenteId · estado · resueltaPorDocenteId? · resueltaEn? · motivoResolucion? · actualizadoEn

Índices: `por_matricula_periodo` [matriculaId, periodoAcademicoId, estado] · `por_matricula_fecha` [matriculaId, fechaOcurrancia]

puntajePeriodo

matriculaId · periodoAcademicoId · puntajeBase · puntosPositivos · puntosNegativos · puntajeActual · franjaConductaId? · congelado · recalculadoEn

Índice: `por_matricula_periodo` [matriculaId, periodoAcademicoId]

registroAsistencia

matriculaId · periodoAcademicoId · fecha · estado · observacion? · registradoPorDocenteId · justificadaEn? · actualizadoEn

Índice: `por_matricula_fecha` [matriculaId, fecha]

reporteGeneral · reporteGeneralValor

reporteGeneral: cursoId · periodoAcademicoId · plantillaReporteId · fecha · estado · publicadoEn? · publicadoPorDocenteId? · actualizadoEn — índice `por_curso_fecha`

reporteGeneralValor: reporteGeneralId · plantillaCampoId · valorTexto? — índice `por_reporte_campo`

reporteEstudiante · reporteEstudianteItem · entregaReporte

reporteEstudiante: matriculaId · periodoAcademicoId · reporteGeneralId? · fecha · tieneNovedades · puntajeAlCierre · franjaConductaId? · estadoAsistencia? · generadoEn — índice `por_matricula_fecha`

reporteEstudianteItem: reporteEstudianteId · tipoItem · accionRegistradaId? · registroAsistenciaId? · comunicadoCursoId? · textoLibre? · orden — índice `por_reporte`

entregaReporte: reporteEstudianteId · representanteId · entregadoEn? · leídoEn? — índice `por_reporte_representante`

comunicadoCurso

cursoId · periodoAcademicoId? · tipo · alcance · estudianteId? · titulo · contenido · fechaEvento? · horaEvento? · visibleDesde · visibleHasta · creadoPorDocenteId · activo

Índices: `por_curso_ventana` [cursoId, activo, visibleHasta] · `por_estudiante` [estudianteId, activo]

franjaConducta · categoriaAccion · tipoAccion · plantillaReporte · plantillaCampo

Las cinco vienen sembradas por `semillas:cargar`. Las **lees**; solo escribirías en ellas si una institución personalizara su catálogo, que no está en el alcance de la v1.

Apéndice B — Guardas disponibles

Función	Para qué
<code>calcularPuntaje(puntos[], base?, piso?, techo?)</code>	El recálculo de ADR-005, con sus topes
<code>exigirTopeDiario(acumuladoHoy, puntosNuevos)</code>	C4: +4 / -5 por día
<code>exigirSignoCoherente(signo, min, max)</code>	Que una positiva no traiga rango negativo
<code>exigirRangoTipoAccion(min, defecto, max)</code>	Que el defecto caiga dentro del rango
<code>exigirRangoFechas(inicio, fin, entidad)</code>	Que un rango de fechas sea coherente
<code>exigirVentanaComunicado(desde, hasta)</code>	Ventana de visibilidad de un comunicado
<code>exigirDuracionNota(dias)</code>	Nota del profesor: entre 1 y 7 días
<code>exigirFechaEvento(tipo, fechaEvento?)</code>	Un evento trae fecha; una nota no
<code>exigirAlcanceCoherente(alcance, estudianteId?)</code>	CURSO sin estudiante, ESTUDIANTE con él
<code>exigirPublicacionCompleta(...)</code>	Publicar trae fecha y autor
<code>hoyEnGuayaquil(ahora?)</code>	La fecha de hoy en la zona del piloto
<code>sumarDias(fecha, dias)</code>	Aritmética de fechas sin salir de la zona
<code>ErrorDominio(codigo, mensaje)</code>	El error que la app sabe distinguir

Todas en `movil/convex/lib/guardas.ts`. **Léelo entero una vez** — son 260 líneas y te ahorra reescribir media docena de validaciones.

Apéndice C — REGLAS que te tocan

Constante	Valor	Qué es
PUNTAJE_BASE	60	Con lo que arranca cada estudiante en cada parcial
PUNTAJE_MINIMO / MAXIMO	0 / 100	Piso y techo del recálculo
TOPE_DIARIO_POSITIVO	4	C4 — máximo que puede ganar en un día
TOPE_DIARIO_NEGATIVO	5	C4 — máximo que puede perder en un día
POSITIVA_MIN / MAX	1 / 2	C2 — rango de una acción positiva
RESPONSABILIDAD_PUNTOS	-1	C2 — irresponsabilidad, fija
DISCIPLINA_MIN / MAX	-3 / -1	C2 — indisciplina y deshonestidad
REPORTES_PREVIOS_FREE	2	E4 — reportes anteriores, plan gratuito
REPORTES_PREVIOS_PREMIUM	7	E4 — reportes anteriores, premium
NOTA_PROFESOR_DIAS_MIN / MAX	1 / 7	Duración de una nota del profesor
EVENTO_DIAS_ANTICIPACION_MAX	7	Con cuánta anticipación aparece un evento
ZONA_HORARIA	America/Guayaquil	UTC-5 todo el año, sin cambio de hora

Apéndice D — Qué deja `semillas:cargar`

Las 6 franjas de conducta

Código	Nombre	Rango
CRITICA	Situación crítica	0-15
MUY_BAJO	Muy por debajo	16-30
BAJO	Por debajo de lo esperado	31-50
BASE	En el punto de partida	51-60
BUENO	Buen desempeño	61-80
EXCELENTE	Excelente	81-100

Las 6 categorías y los 7 tipos de acción

Categorías: DESEMPEÑO · CONVIVENCIA · RESPONSABILIDAD · PUNTUALIDAD · DISCIPLINA · DESHONESTIDAD

Tipos positivos (+1 a +2): POS_DESEMPEÑO · POS_CONVIVENCIA · POS_RESPONSABILIDAD · POS_PUNTUALIDAD

Tipos negativos: NEG_INDISCIPLINA (-3 a -1) · NEG_DESHONESTIDAD (-3 a -1) · NEG_IRRESPONSABILIDAD (-1 fijo)

La plantilla del reporte

Una plantilla «Reporte diario del curso» con cuatro campos: ANUNCIOS, NOVEDADES, TAREAS (texto largo) y CONSEJO (texto corto).

Apéndice E — Errores que vas a cometer

Síntoma	Causa
«tu cuenta existe pero aún no completaste tu perfil» en una prueba	Sembraste <code>authSubject</code> sin el prefijo <code>https://convex.test </code>
<code>calcularPuntaje</code> devuelve cualquier cosa	Le pasaste la base como primer argumento. El primero es el arreglo de puntos
<code>exigirVinculo</code> no compila	Le pasaste tres argumentos. Son dos: <code>(ctx, estudianteId)</code>
La pantalla dice «no pudimos completar la solicitud» y no el motivo real	Falta envolver el handler en <code>conErroresPublicos</code>
<code>.unique()</code> lanza error	Hay más de una fila. Casi siempre es unicidad que no comprobaste antes de insertar
El puntaje se desincroniza	Sumaste en vez de recalcular. Busca cualquier <code>puntaje ± n</code>
Un reporte cambia después de publicado	Estás leyendo las acciones al vuelo en vez de la fotografía guardada
El CI falla y en tu máquina no	Un archivo sin subir, casi siempre <code>_generated/api.d.ts</code>